

Assignment: Steady-State Error

This assignment exercises the steady-state error analysis from the chapter. All computations assume *unity negative feedback* and a *stable* closed loop unless stated otherwise. MATLAB is encouraged throughout as a verification tool — a sample script is in `assignment_solutions.m`.

Exercise 1 — Three Test Inputs

The unity-feedback system has open-loop transfer function

$$G(s) = \frac{600}{(s+10)(s^2+4s+25)}.$$

For example, $G(s)$ might represent a spacecraft attitude-control loop: a reaction-wheel actuator (the real pole at $s = -10$) driving a rigid spacecraft bus coupled to a lightly-damped flexible solar panel (the complex pole pair) — recall from the chapter that $G(s)$ here is the *combined* open-loop $C(s)P(s)$, not the plant alone.

- Identify the system Type by inspection of $G(s)$.
- Find the steady-state error for the inputs $u(t)$, $tu(t)$, and $\frac{1}{2}t^2u(t)$. (The factor of $\frac{1}{2}$ on the parabola is just a bookkeeping convenience: it gives $\mathcal{L}\{\frac{1}{2}t^2\} = 1/s^3$, which keeps the algebra clean.)
- In MATLAB, verify your step-input answer in (b) two ways:
 - Calculate: compute the closed-loop transfer function and evaluate its DC gain.
 - Simulate: generate the step response and check the final value of the error signal.
- Two of your answers in (b) are infinite. In one sentence, explain what physical fact about $G(s)$ makes that so.

Exercise 2 — Design a P-Only Velocity Tracking Compensator

You will design a proportional-only controller for a position-control plant. The plant is

$$G_p(s) = \frac{s+5}{s(s+8)(s+12)},$$

and the controller is $C(s) = K_p$. The system is closed with unity feedback.

For example, $G_p(s)$ might model a machine-tool feed drive, where accurate constant feed velocity is required: the integrator $1/s$ converts the motor's angular velocity to linear position, the real poles are the motor's electrical and mechanical time constants, and the zero comes from a tachometer-feedback element built into the plant.

Keep track of which transfer function you are working with at each step: the *open-loop*, *forward path* or the *closed loop*.

- What is the system Type?
- Find the value of K_p such that the steady-state error to a unit ramp input is 4%.

- (c) Use MATLAB to compute the closed-loop poles for your value of K_P . Are they all in the left half-plane?

Exercise 3 — USV Surge: Predict Before You Simulate

The USV surge (forward-speed) plant, identified from the lab data, is

$$G_{\text{surge}}(s) = \frac{5.1}{1.3s + 1}.$$

For each controller architecture below, find the steady-state error to a unit-step (1 m/s) speed command. Use analytical methods, MATLAB, or both.

- (a) **P only:** $K_P = 0.5$.
- (b) **PI:** $K_P = 0.5$, $K_I = 0.4$.
- (c) **P + feedforward:** $K_P = 0.5$, $K_{ff} = 1/K_{\text{plant}}$.
- (d) **PI + feedforward:** same K_P, K_I as (b), and K_{ff} as (c).
- (e) Use the *adaptation argument* from the chapter (not the steady-state error formula) to explain in one or two sentences why the PI controller in (b) drives the error to zero.

Exercise 4 — USV Yaw: Ramp Tracking Design

The USV's yaw-rate plant, identified from the lab data, is

$$G_{\text{yaw-rate}}(s) = \frac{0.82}{0.61s + 1}.$$

We want to control *heading angle*, not yaw rate, to follow a course. Heading is the integral of yaw rate. A proportional controller $C(s) = K_r$ is used in a unity-feedback heading loop. The heading reference (the command) ramps at $5^\circ/\text{s}$ (a constant turn-rate command).

- (a) Write the plant transfer function from rudder input to *heading angle*, $G_r(s)$. (Hint: heading is the integral of yaw rate, so $G_r(s)$ picks up an extra factor relative to the yaw-rate plant above.)
- (b) What is the system Type of the closed loop? Justify in one phrase.
- (c) Write the steady-state heading error in degrees as a function of K_r .
- (d) The design spec is an absolute heading error $\leq 1^\circ$. Find the minimum K_r that meets the spec.
- (e) Verify your K_r in MATLAB by simulating the closed-loop ramp response and checking the steady-state error.